

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

COMP4128 Programming Challenges

School of Computer Science and Engineering
UNSW Sydney

Term 2, 2026

Extension

More on
Range Trees
RUPQ
RURQ
Range Trees
of DS
Searching a
Range Tree
DP
Optimizations
CHT
Construction
Application
Examples
D&C
Framework
Monotonicity
Modifications

- 1 More on Range Trees
 - Range Updates, Point Queries
 - Range Updates, Range Queries
 - Range Trees of Data Structures
 - Searching a Range Tree
- 2 DP Optimizations
 - Convex Hull Trick
 - Construction
 - Application
 - Examples
 - Divide and Conquer Optimization
 - Divide and Conquer Framework
 - Proving monotonicity of opt
 - Modifications

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- So far, our range trees have supported range queries and point updates.
- We will now extend this to also support range updates.
- For simplicity, let us tackle the problem of range updates, point queries first.

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- Given an array $a[n]$, initially all zeros, support q operations, each being one of the following forms:
 - **Update:** $U\ l\ r\ v$. Perform $a[1,r) += v$.
 - **Query:** $Q\ x$. Output $a[x]$.
- $n, q \leq 100,000$.

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- Again, we can't just individually update all elements of the array: that would cost $O(n)$ per operation.
- So we are going to do the same as we did for range queries.
- Suppose our update tells us to perform $a[1,r) += x$.
- This is the same as performing $a[1,m) += x$ and $a[m,r) += x$.
- So we can partition our initial update into smaller ranges however we wish.

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

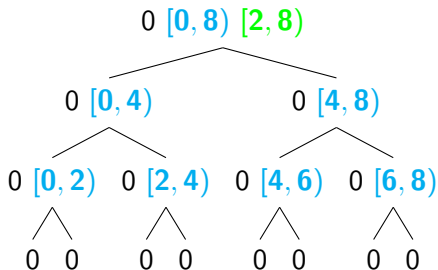
Monotonicity

Modifications

- We will decompose $[l, r)$ into ranges that correspond directly to nodes in our range tree in the same way that we do for range queries.
- For each node we will store a *lazy counter* that keeps the sum of all updates to that node's range of responsibility.
- To query an index, we need to know all updates to ranges that contain said index.
- For a range tree there are $O(\log n)$ of these ranges, and they are exactly the ranges that appear on the path from the root to the leaf corresponding to that index.

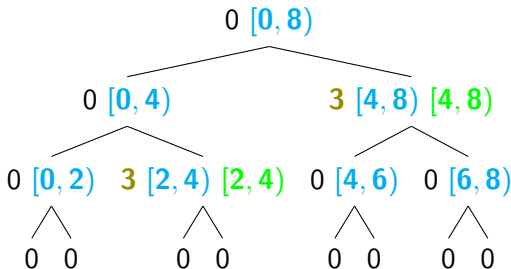
Extension

- Let's update the range $[2, 8)$ with $v = 3$.



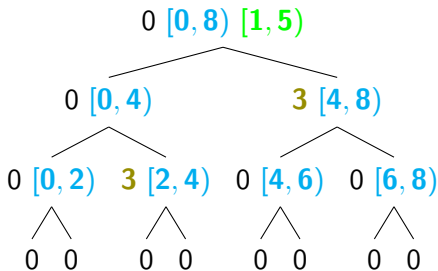
Extension

- As with range queries, we will push the update range down into the applicable nodes.



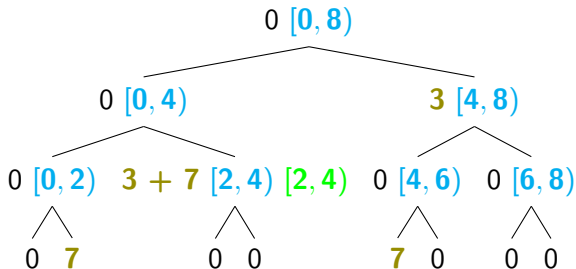
Extension

- Now let's assume we've been given a second update, for the range $[1, 5)$ with $v = 7$.



Extension

- The range decomposition is $[1, 2)$, $[2, 4)$, $[4, 5)$.



More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

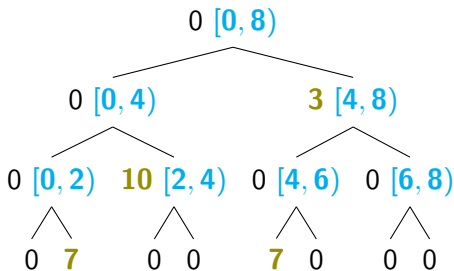
Framework

Monotonicity

Modifications

Extension

- Note that we have not changed the value for the node corresponding to the range $[4, 8)$.



More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DSSearching a
Range TreeDP
Optimizations

CHT

Construction

Application

Examples

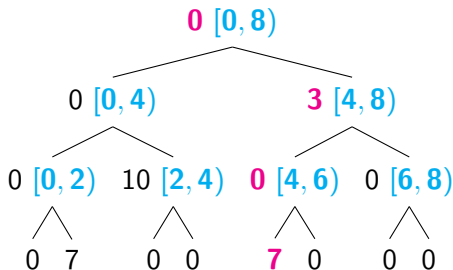
D&C

Framework

Monotonicity

Modifications

- Let's try to query what $a[4]$ is.
- The nodes responsible for a range containing $i = 4$ are the ones from the leaf for i to the root.



- The sum of these is $0 + 3 + 0 + 7 = 10$, hence $a[4] = 10$.

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

```
const int N = 100100;
long long lazyadd[1<<18];

int n;

// The root node is responsible for [0, n). Update range [uL, uR)
// Compare to range query code.
void update(int uL, int uR, int v, int i = 1, int cL = 0, int cR = n) {
    if (uL == cL && uR == cR) {
        lazyadd[i] += v;
        return;
    }
    int mid = (cL + cR) / 2;
    if (uL < mid) update(uL, min(uR, mid), v, i * 2, cL, mid);
    if (uR > mid) update(max(uL, mid), uR, v, i * 2 + 1, mid, cR);
}

long long query(int p, int i = 1, int cL = 0, int cR = n) {
    if (cR - cL == 1) {
        return lazyadd[i];
    }
    int mid = (cL + cR) / 2;
    long long ans = lazyadd[i];
    if (p < mid) ans += query(p, i * 2, cL, mid);
    else ans += query(p, i * 2 + 1, mid, cR);
    return ans;
}
```

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- **Complexity:** Still $O(\log n)$ per operation, for the same reasons as before.
- Works for most operations that can be broken down into smaller ranges.
- You also need to be able to accumulate the operation so that you can store all the information in the *lazy counter*. So for example, the operation $a[i] = a[i] \bmod v_q$ is an issue.
- However, covers most operations you would naturally think of, e.g: multiply, divide, xor, and, etc ...
- Can also sometimes do multiple different kinds of updates but this is more finicky and depends on the specific updates and probably requires lazy propagation.

Extension

- **Problem Statement** Given a weighted tree, all edges initially 0. Support q operations, each one taking one of the following forms:
 - **Update** $U\ i\ j\ w$: Add w to the weight of the edge between i and j .
 - **Query** $Q\ i\ j$: Output the shortest distance between i and j .
- **Input** A tree described as $n - 1$ edges, followed by q operations. $1 \leq n, q \leq 100,000$.
- **Output** For each query, an integer, the shortest distance from i to j .

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

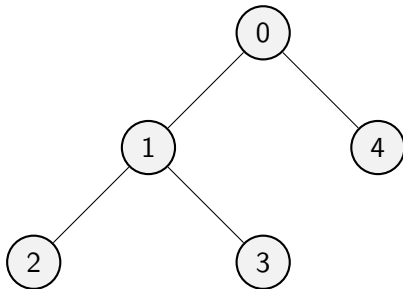
Examples

D&C

Framework

Monotonicity

Modifications



Sample Input:

U 0 1 3

Q 4 3

U 0 1 -1

U 3 1 5

Q 3 4

Sample Output:

3

7

Extension

- Recall our solution for the static problem.
- Let $l = \text{lca}(i, j)$. We split the path from i to j into a path from i to l followed by a path from j to l .
- Let $\text{weight_sum}(i)$ be the sum of weights from the root to i . The answer is then just $\text{weight_sum}(i) + \text{weight_sum}(j) - 2 * \text{weight_sum}(l)$.
- Our updates don't change the tree structure but change weight_sum . So this is what we need to update.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- When we update an edge, what weight sums do we update?
- Every node whose path to the root goes through said edge. In other words, every node in the edge's subtree.
- So we should maintain `weight_sum` using a range tree and update it using subtree updates.

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

```
using namespace std;

const int N = 100100;

// Suppose you already have your tree set up.
int depth[N]; // Depth in tree (ignores weight).
int lca(int a, int b);

// A node is responsible for the range [startRange[v], endRange[v]]
int indexInRangeTree[N], startRange[N], endRange[N];
// A range tree supporting range updates of add, point queries of value.
long long rangeTree[1<<18];
void update(int uL, int uR, long long v); // value[uL,uR) += v
long long query(int q);

void update_edge(int i, int j, long long v) {
    // WLOG i is parent, j is child
    if (depth[i] > depth[j]) { swap(i, j); }
    update(startRange[j], endRange[j], v);
}

long long get_tree_distance(int i, int j) {
    int l = lca(i, j);
    return query(indexInRangeTree[i]) + query(indexInRangeTree[j])
        - 2*query(indexInRangeTree[l]);
}
```

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- Now let's try range updates and range queries.

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- Given an array $a[n]$, initially all zeros, support q operations, each being one of the following forms:
 - Update:** $U \ l \ r \ v$. Perform $a[1, r) += v$.
 - Query:** $Q \ l \ r$. Output $\sum_{i=l}^{r-1} a[i]$.
- $n, q \leq 100,000$.

Extension

- We will support range updates in the same way we did for point queries. Instead, we will change how we do range queries.
- In our earlier example, for each node we just stored the lazy counter. This was enough as every query involved walking from the root to a leaf.
- However, recall to handle range queries in good time complexity we terminate our recursion once we've found a node that matches our current query range.
- Hence for each node we will need to store 2 values, the lazy counter **and** the sum of the node's range of responsibility.
- 2 major changes:
 - ① Maintain for each node its lazy counter and the sum of its range.
 - ② Support updates through *lazy propagation*.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- *Lazy propagation* is the idea that whenever we touch a node, we should propagate that node's updates to its children.
- For our example, propagate means add the lazy counter of node i to its two children and set the lazy counter of node i to 0.
- Essentially, instead of doing $a[1,r) += v$, we break the update into $a[1,m) += v$ and $a[m,r) += v$.

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

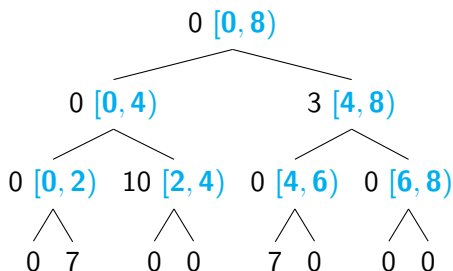
Examples

D&C

Framework

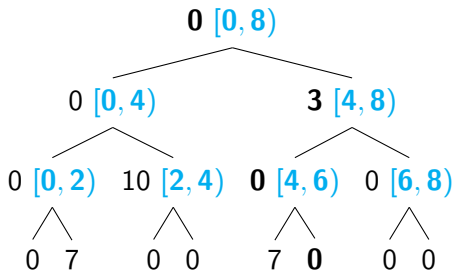
Monotonicity

Modifications



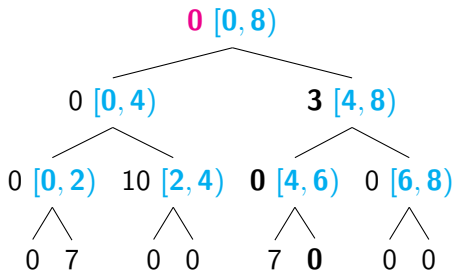
Extension

- Let's try querying $a[5]$.



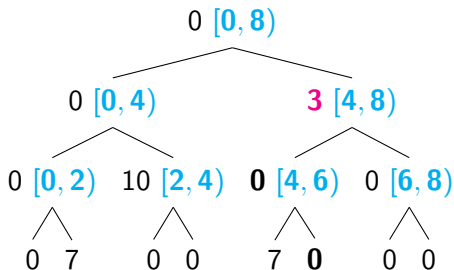
Extension

- Let's try querying $a[5]$.



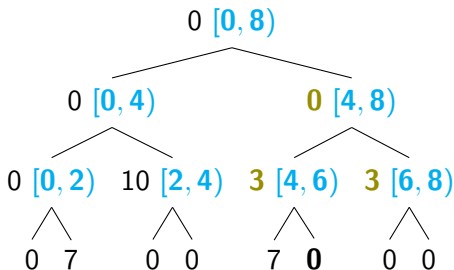
Extension

- Let's try querying $a[5]$.



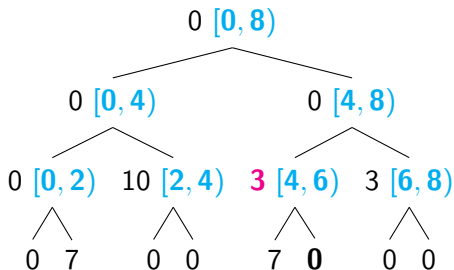
Extension

- Let's try querying $a[5]$.



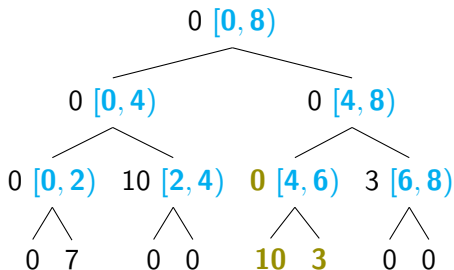
Extension

- Let's try querying $a[5]$.



Extension

- Let's try querying $a[5]$.



More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

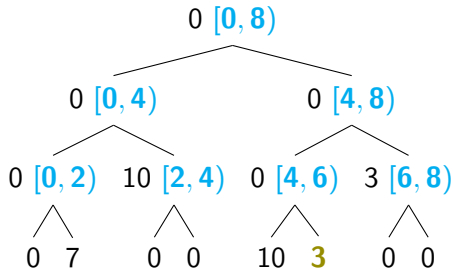
Framework

Monotonicity

Modifications

Extension

- Let's try querying $a[5]$.



- Hence $a[5] = 3$.

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- This ensures when we read the sum of a range from a node, we won't be missing any updates that are stored in the lazy counter of one of the node's ancestors.
- **Complexity Overhead?** No overhead, propagation is an $O(1)$ operation per node.

Extension

- To support range queries, for each node we also need to store the sum of its range.
- But because of range updates, we can't literally do this (else we'd need to update every node within the update range).
- Our invariant will be: Each node stores what the sum of its range would be, accounting only for lazy counters within its subtree.
- All lazy counters above each node are ignored.
- This way, an update only needs to modify the nodes encountered in the update's recursion.
- This will suffice since lazy propagation ensures when we actually query a node all its ancestors will have lazy counter 0.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

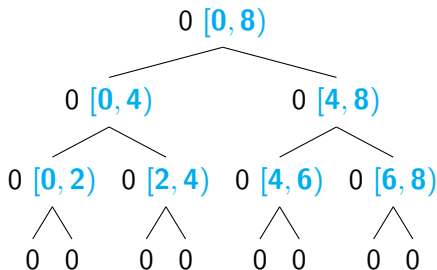
Framework

Monotonicity

Modifications

Extension

- Sum of a node's range will always be shown.
- Nonzero lazy counters will be written in brackets to the right.



More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

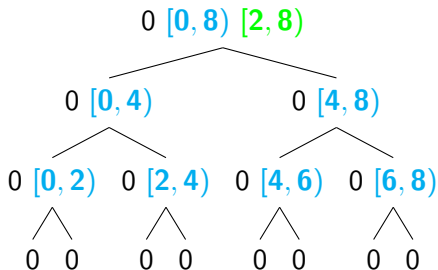
Framework

Monotonicity

Modifications

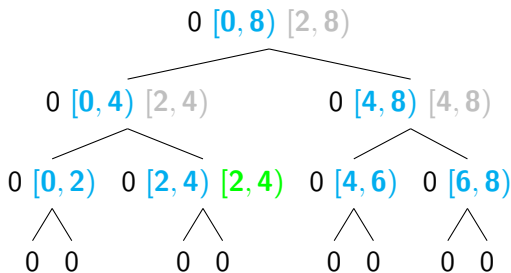
Extension

- Let's update the range $[2, 8)$ with $v = 3$.



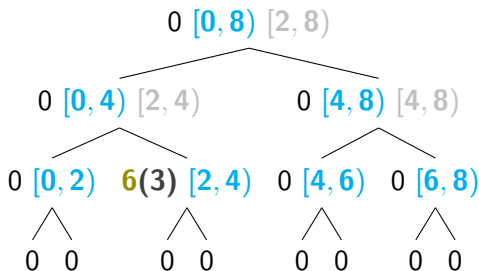
Extension

- This is done recursively, just like queries, so we'll summarise.



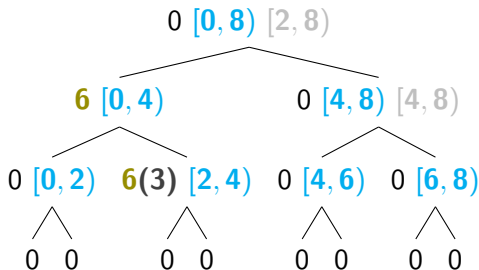
Extension

- Let's update the left side first.
- We need to update the lazy counter and also the sum.



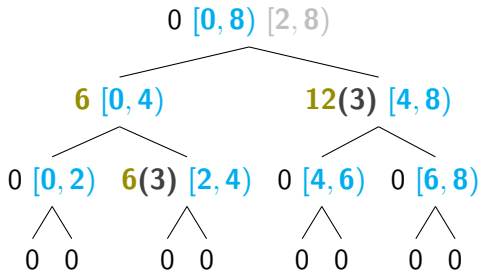
Extension

- We then return from this branch of the recursion.
- As we're returning we will update the nodes we passed through in this branch.



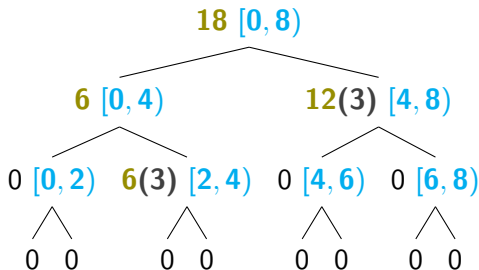
Extension

- Now our recursion enters the other branch. Same as before.



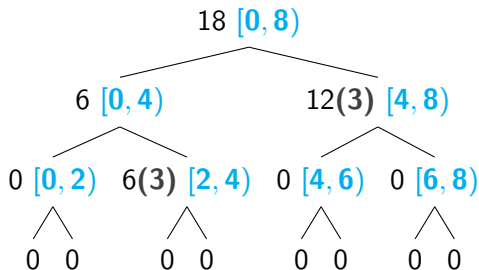
Extension

- We now return from the right branch.
- We now update the root node before returning.



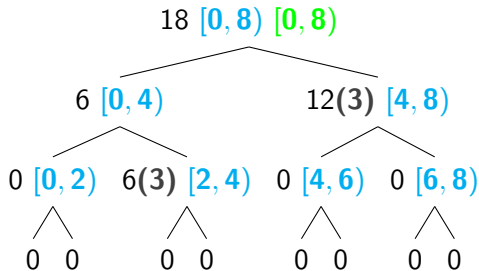
Extension

- We now return from the right branch.
- We now update the root node before returning.



Extension

- Let's update a second update to the range $[0, 8)$ with $v = 4$.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DSSearching a
Range TreeDP
Optimizations

CHT

Construction

Application

Examples

D&C

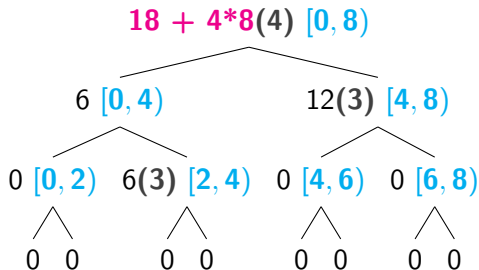
Framework

Monotonicity

Modifications

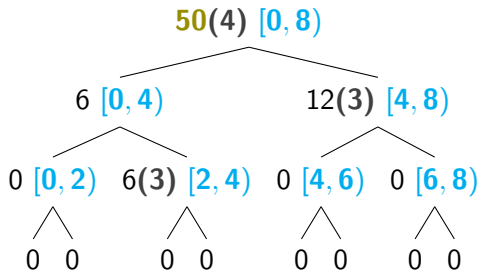
Extension

- Since the root's range matches we just update the root.



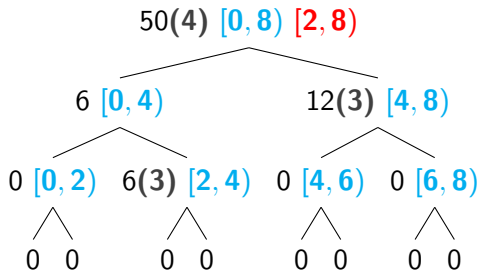
Extension

- Note how we did not modify any node but the root.



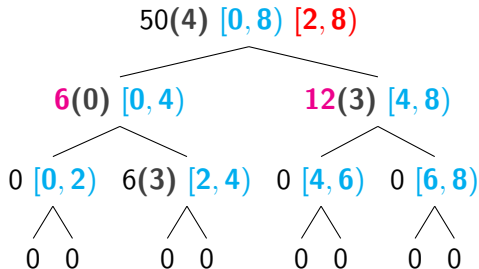
Extension

- Let's now query the sum of the range $[2, 8)$.



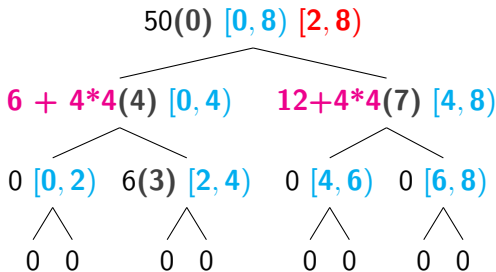
Extension

- Whenever we encounter a node, we lazy propagate out its lazy counter.



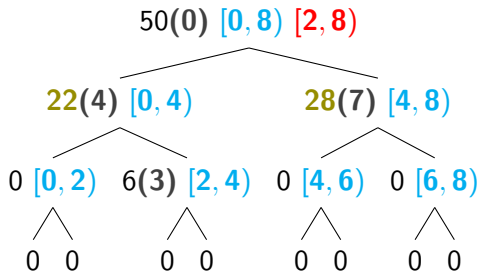
Extension

- When we lazy propagate, we also need to change node sums.



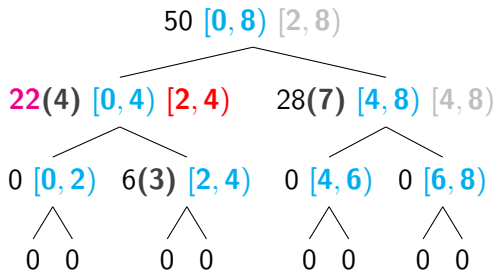
Extension

- When we lazy propagate, we also need to change node sums.



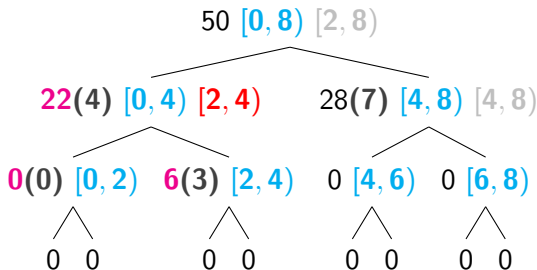
Extension

- Now we do more recursion for answering the query.



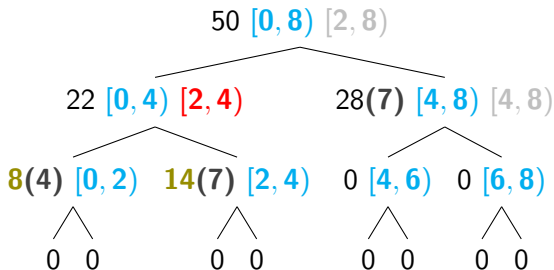
Extension

- Again, we need to lazy propagate.



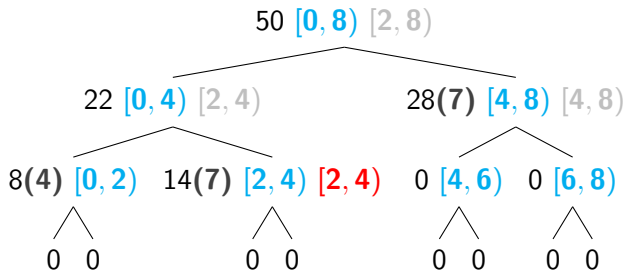
Extension

- Again, we need to lazy propagate.



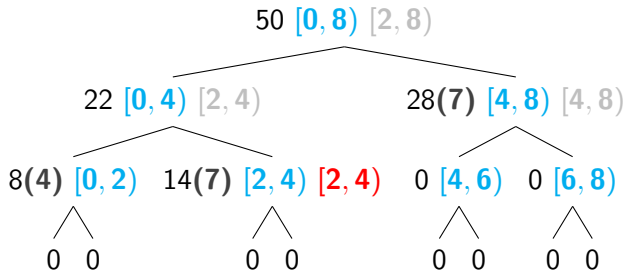
Extension

- Now we recurse again.



Extension

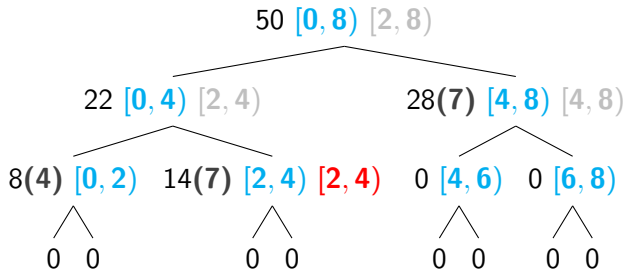
- For simplicity, we'll just say we don't lazy propagate when we've found the right range.



- So we return the result we have obtained up the chain and continue the query in the other branch.

Extension

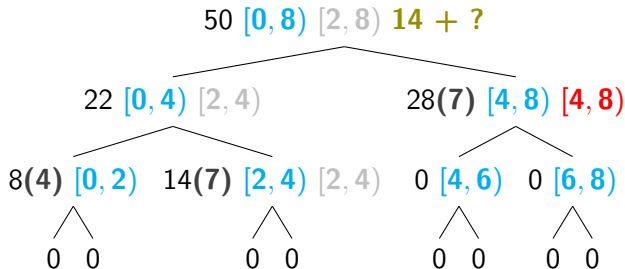
- So we return the result we have obtained up the chain and continue the query in the other branch.



- Note how all ancestors of the node responsible for the range $[2, 4)$ have lazy counter equal to 0.

Extension

- Now we continue in the second branch where we immediately find the node with the right range.



More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

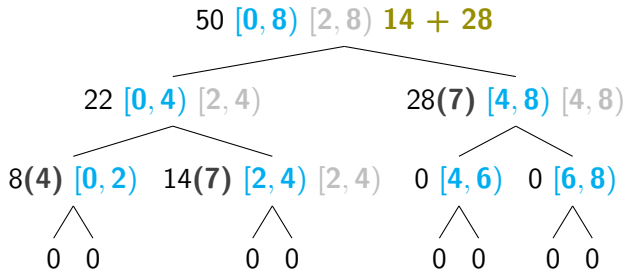
Framework

Monotonicity

Modifications

Extension

- So we immediately return with the value.



- And we now return from the root with the answer 42.

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- Implementation wise, it helps to introduce some terminology.
- In a recursion, call the “preorder procedure” the procedure we call before recursing.
- Call the “postorder procedure” the procedure we call after we’ve returned from all children.
- Then we will implement propagation as a preorder procedure.
- And for updates, recalculating a node’s sum is a postorder procedure.

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

```
using namespace std;

const int N = 100100;
long long lazyadd[1<<18], sum[1<<18];

// Procedure for recalculating a node's sum from its lazy and children.
void recalculate(int id, long long l, long long r) {
    sum[id] = lazyadd[id] * (r - l);
    if (r - l != 1) {
        sum[id] += sum[id * 2];
        sum[id] += sum[id * 2 + 1];
    }
}

void update_lazy(int id, long long v, long long l, long long r) {
    lazyadd[id] += v;
    recalculate(id, l, r);
}

// Preorder procedure for propagation. Do NOT call it on leaves.
void propagate(int id, long long l, long long r) {
    long long mid = (l + r) / 2;
    update_lazy(id * 2, lazyadd[id], l, mid);
    update_lazy(id * 2 + 1, lazyadd[id], mid, r);
    lazyadd[id] = 0;
}
```

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

```
int n;

// The root node is responsible for [0, n). Update range [uL, uR)
void update(int uL, int uR, int v, int i = 1, int cL = 0, int cR = n) {
    if (uL == cL && uR == cR) {
        update_lazy(i, v, cL, cR);
        return;
    }
    propagate(i, cL, cR);
    int mid = (cL + cR) / 2;
    if (uL < mid) update(uL, min(uR, mid), v, i * 2, cL, mid);
    if (uR > mid) update(max(uL, mid), uR, v, i * 2 + 1, mid, cR);
    recalculate(i, cL, cR);
}

long long query(int qL, int qR, int i = 1, int cL = 0, int cR = n) {
    if (qL == cL && qR == cR) {
        return sum[i];
    }
    propagate(i, cL, cR);
    int mid = (cL + cR) / 2;
    long long ans = 0;
    if (qL < mid) ans += query(qL, min(qR, mid), i * 2, cL, mid);
    if (qR > mid) ans += query(max(qL, mid), qR, i * 2 + 1, mid, cR);
    return ans;
}
```

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- **Complexity:** $O(\log n)$ per update/query still. We still visit the same nodes; the extra propagation and computation is just $O(1)$ overhead per node.
- It is important to make sure you have invariants in mind when implementing range trees.
- For example, we had the invariant that `sum[i]` represents the sum accounting for all lazy updates in the subtree of i . Everything else was dictated by maintaining this invariant.
- You could instead have `sum[i]` account for all lazy updates in the subtree, excluding the lazy counter at node i itself.
- Doesn't matter, just stay consistent.

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- **Problem Statement:** Given an array of integers $a[n]$, initially all 0, support q operations of the forms:
 - **Update** $U \ l \ r \ v$. Set $a[l, r) = v, v \geq 0$.
 - **Query** $Q \ l \ r$. What is the max of $a[l, r)$?
- **Input Format:** First line, 2 integers, n, q . The following q lines each describe an operation.
- **Constraints:** $1 \leq n, q \leq 100,000$.

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Sample Input:

5 7

U 1 3 5

U 2 4 1

Q 1 3

Q 2 3

U 3 4 3

Q 2 4

Q 1 5

Sample Output:

5

1

3

5

Extension

- We will use the same lazy propagation framework.
- What our nodes store is dictated by the queries.
- Each of our nodes needs to store the max for their range of responsibility, *ignoring* all lazy values outside that node's subtree.
- Our lazy values are dictated by the updates.
- Each of our nodes needs to store the last update applied to the node.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- **Question?** For a given node in the range tree how do we know which update most recently covered the node's range?
- **Key Observation:** If we lazy propagate, it is the lazy value of the highest ancestor with a lazy value set.
- **Why?** Because whenever we apply an update, we lazy propagate existing updates on the path to the node we're updating. So no ancestors of the node have lazy values set. Hence the highest set lazy value is the most recent update.

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- Now we know what we need.
- Our lazy values store the most recent update to a range. These will be lazy propagated. When we lazy propagate we just overwrite our children since we know our update is more recent than our children's.
- Each node stores the max of its range, based on only lazy values within its subtree.
- $\text{maxrt}[i] = \text{lazy}[i]$ if $\text{lazy}[i]$ is set, else it is the max of i 's children.

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT
Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

```
#include <algorithm>
#include <iostream>
using namespace std;

const int N = 100100;
const int UNSET = -1;
// Since A is 0 initially, the default values are correct.
int lazyset[1<<18]; // UNSET if no lazy is set
int maxrt[1<<18];

// Recalculates a node's values assuming its children are correct.
// do NOT call these on leaves.
void recalculate(int i) {
    if (lazyset[i] != UNSET) // should never happen
        maxrt[i] = lazyset[i];
    else
        maxrt[i] = max(maxrt[i*2], maxrt[i*2+1]);
}

void propagate(int i) {
    if (lazyset[i] == UNSET)
        return;
    lazyset[i*2] = lazyset[i*2+1] = lazyset[i];
    maxrt[i*2] = maxrt[i*2+1] = lazyset[i];
    lazyset[i] = UNSET;
}
```

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

```
int n;

void update(int uL, int uR, int v, int i = 1, int cL = 0, int cR = n) {
    if (uL == cL && uR == cR) {
        lazyset[i] = maxrt[i] = v;
        return;
    }
    propagate(i);
    int mid = (cL + cR) / 2;
    if (uL < mid) update(uL, min(uR, mid), v, i*2, cL, mid);
    if (uR > mid) update(max(uL, mid), uR, v, i*2+1, mid, cR);
    recalculate(i);
}

int query(int qL, int qR, int i = 1, int cL = 0, int cR = n) {
    if (qL == cL && qR == cR) {
        return maxrt[i];
    }
    propagate(i);
    int mid = (cL + cR) / 2;
    int ans = -1; // note all values are >= 0 in the question.
    if (qL < mid) ans = max(ans, query(qL, min(qR, mid), i*2, cL, mid));
    if (qR > mid) ans = max(ans, query(max(qL, mid), qR, i*2+1, mid, cR));
    return ans;
}
```

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

```
int main (void) {
    int n, q;
    cin >> n >> q;

    // not actually necessary
    // a prior update of 0 to [0,n) would have no effect
    fill(lazyset, lazyset+(1<<18), UNSET);

    for (int i = 0; i < q; i++) {
        char type;
        cin >> type;
        if (type == 'U') {
            int l, r, v;
            cin >> l >> r >> v;
            update(l, r, v);
        } else if (type == 'Q') {
            int l, r;
            cin >> l >> r;
            cout << query(l,r) << '\n';
        }
    }
}
```

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- So far we've just used range trees to support operations on an array of integers.
- But the real power of range trees is in the way it decomposes ranges.
- The nodes can store anything.
- For example other data structures (!!)
- The most useful is probably a set or other SBBST.

Extension

More on
Range Trees
RUPQ
RURQ
Range Trees
of DS
Searching a
Range Tree
DP
Optimizations
CHT
Construction
Application
Examples
D&C
Framework
Monotonicity
Modifications

- **Problem Statement** It's 2200 and climatology is now the most hectic job on earth. There is a constant deluge of rain predictions concerning the towns in LineLand. There are n towns in LineLand, in a line. Each prediction is of the form $U \ l \ r \ d$ saying that there will be rain in towns $[l, r)$ on day d . Interspersed among these updates, there will be queries of the form $Q \ a \ d$, asking if there is a predicted shower in town a on day d .
- **Input** First line, n, q , the number of towns and operations. $1 \leq n, q \leq 100,000$. Towns are 0 indexed. The next q lines are the operations in the specified format.
- **Output** For each operation, 1 if there is forecasted rain and 0 otherwise.

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Sample Input:

```
10 6
U 0 3 1
Q 1 1
Q 1 2
Q 3 1
U 1 4 1
Q 3 1
```

Sample Output:

```
1
0
0
1
```

Extension

- We have the characteristic range updates that suggest **range** tree.
- But it no longer suffices to store a single integer for each range.
- To store our predictions we should use a set.
- We will decompose the range of each prediction using the range tree and update the sets of each of the corresponding nodes.
- To answer a query, we need the predictions corresponding to each range containing the queried city. This is just the nodes on the path from the leaf to the root.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Problem: Should I bring an Umbrella?

73

Extension

```
#include <set>
using namespace std;

const int N = 100100;
set<int> rt[1<<18];
int n;

// The root node is responsible for [0, MAX_N). Update range [uL, uR)
void update(int uL, int uR, int v, int i = 1, int cL = 0, int cR = n) {
    if (uL == cL && uR == cR) {
        rt[i].insert(v);
        return;
    }
    int mid = (cL + cR) / 2;
    if (uL < mid) update(uL, min(uR, mid), v, i * 2, cL, mid);
    if (uR > mid) update(max(uL, mid), uR, v, i * 2 + 1, mid, cR);
}

// Does it rain in index qP on day qD?
bool query(int qP, int qD, int i = 1, int cL = 0, int cR = n) {
    if (rt[i].find(qD) != rt[i].end())
        return true;
    if (cR - cL == 1)
        return false;
    int mid = (cL + cR) / 2;
    if (qP < mid) return query(qP, qD, i * 2, cL, mid);
    else return query(qP, qD, i * 2 + 1, mid, cR);
}
```

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- **Complexity?** $O(n + q \log^2 n)$. Each update and query accesses $O(\log n)$ nodes (this is a characteristic of the range decomposition itself) but each access costs $O(\log n)$ due to the sets.
- **Warning:** We can't lazy propagate in this example. This is because the size of the data we are storing at each node isn't constant any more. So the cost of lazy propagation per operation is potentially $O(n \log n)$ and this does not amortize.
- E.g: have 50000 updates to the entire range, then have the next 50000 be queries forcing a $O(n \log n)$ set copy each time.

Extension

- We can actually do much more.
- We can support other things sets and maps and OSTs support, like deleting predictions and finding the closest rain day or counting the number of cities raining on a given day in a range.
- If the bounds were different (fewer cities) we could even support updates affecting ranges of days, by storing a range tree of days in each node of the original range tree.
- **Moral:** If you need to store different kinds of data while supporting range operations, consider a range tree of a suitable data structure.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- Another classic problem, finding total area covered by a set of rectangles.

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- **Problem Statement:** It's 2201 and you're done with Earth and its unpredictable rainfall. You've decided to move to Neptune. After landing, you find out, to your dismay, that not only does it rain on Neptune but it rains diamonds. But it's too late now to turn back so you'll just have to make do.

As we all know, Neptune is a $n \times n$ grid with bottom left corner $(0,0)$. There are m diamond showers on Neptune, each a rectangle. You now wish to find how much of Neptune is covered by diamond showers.

- **Input:** First line 2 integers, n, m . $1 \leq n, m \leq 100,000$. The next m lines are each of the form $x_0 \ y_0 \ x_1 \ y_1$, describing a diamond shower with bottom left corner (x_0, y_0) and upper right corner (x_1, y_1) .
- **Output:** A single integer, the total area of Neptune covered by the union of all the showers.

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

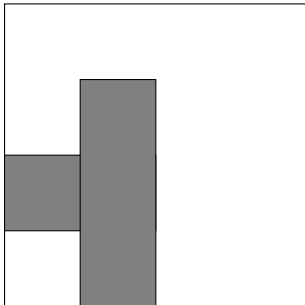
Sample Input:

```
4 2
0 1 2 2
1 0 2 3
```

Sample Output:

4

Explanation:



Extension

- 2 common approaches for 2D problems. Either a 2D data structure or a linear sweep in the y direction while maintaining a data structure over x .
- Latter is generally faster and easier.
- For each row, what do we need to track?
- Which columns currently have a rectangle.
- Standard way of doing this is create 2 events per rectangle, one at y_0 instructing us to activate the rectangle, one at y_1 instructing us to deactivate the rectangle.
- Suppose we have done this so we know which rectangles are active. How do we track how many columns have a rectangle?

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

- An active rectangle covers a *range* of x coordinates so ...*range tree*!
- The query we need to support is count the number of indices that are covered.
- We need to support the updates:
 - 1 Add a range.
 - 2 Remove a range.
- So we have a range update, range query situation.
- What do our nodes store and what are the lazy counters?

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- What our nodes store is dictated by the queries.
- Each of our nodes needs to store the number of covered indices in its range.
- Our lazy counters are dictated by the updates.
- Each of our nodes needs to store whether a range fully covers that node's range.
- We can use a set for the lazy counter. Or we can use a counter.
- **Warning:** We can't lazy propagate here. Else deleting a range becomes a nuisance (this becomes more natural if one thinks of the lazy counter as a set)

Extension

- So we decompose each update range same as how we always do.
- After decomposing the range, we update the lazy counter at the corresponding nodes.
- In addition, each node stores $\text{freq}[i]$, the number of covered indices in its range of responsibility **only** accounting for lazy counters in its subtree.
- Then $\text{freq}[i] = \text{endRange}[i] - \text{startRange}[i]$ if $\text{lazy}[i] > 0$, otherwise, $\text{freq}[i]$ is the sum of its two children.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

```
#include <iostream>
#include <vector>
using namespace std;

const int N = 100100;
// Range tree
int lazycount[1<<18], freq[1<<18];
int n, m;

void recompute(int i, int left, int right) {
    if (lazycount[i] > 0) freq[i] = right-left; // range directly covered
    else if (right-left == 1) freq[i] = 0; // leaf
    else freq[i] = freq[i*2] + freq[i*2+1]; // sum of children
}

// Update count of [uL, uR) by v
void update(int uL, int uR, int v, int i = 1, int cL = 0, int cR = n) {
    if (uL == cL && uR == cR) {
        lazycount[i] += v;
        recompute(i, cL, cR);
        return;
    }
    int mid = (cL + cR) / 2;
    if (uL < mid) update(uL, min(uR, mid), v, i*2, cL, mid);
    if (uR > mid) update(max(uL, mid), uR, v, i*2+1, mid, cR);
    recompute(i, cL, cR);
}

int query_total() {
    return freq[1];
}
```

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

```
struct Event {
    int l, r, v;
    Event(int _l, int _r, int _v) : l(_l), r(_r), v(_v) {}
};

// Convention: process events for a y before calculating that value of y.
// When calculating yi, we will count covered squares in [yi, yi+1]
vector<Event> events[N];

int main() {
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int x0, y0, x1, y1;
        cin >> x0 >> y0 >> x1 >> y1;
        events[y0].emplace_back(x0, x1, 1);
        events[y1].emplace_back(x0, x1, -1);
    }

    long long ans = 0;
    for (int i = 0; i < n; i++) {
        for (const auto &e: events[i])
            update(e.l, e.r, e.v);
        ans += query_total();
    }

    cout << ans << '\n';
}
```

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- For most data structures it suffices to treat them as a black box.
- Hopefully by now you've gotten the sense that this is less true for range trees.
- Sometimes it is useful to also modify how we traverse a range tree.
- This is mainly useful when we are searching for the first/any value that satisfies some given constraint.

Extension

- Let's say we want to find any value that satisfies a criterion X .
- For concreteness, let's say we want to find any value that's at least L .
- In each node, we store enough data to determine if there is a value in its range that satisfies X .
- For our example, we can store the max of all values in each range.
- Once we have this, finding a value is easy. We know for both children whether there is a value inside their range that satisfies X . We then just recurse into whichever side has a value that satisfies X .
- To find the leftmost/rightmost such value, we just bias our search towards the left or right child.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

- Suppose now we want to find if any value in a given range $[l, r)$ that satisfies criterion X .
- Now we just decompose $[l, r)$ into $O(\log n)$ ranges as we usually do with a range tree.
- We can then just repeat this for each of the nodes in our decomposition.
- **Complexity?** $O(\log n)$ if you implement correctly since we actually only need to do this once, to the first node which we know contains a value satisfying X .
- Again, easy to find leftmost/rightmost.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

- **Problem Statement:** Given an array, $a[n]$, all initially 0. Support q operations of the forms:
 - **Update** $U \ i \ v$. Set $a[i] = v$.
 - **Query** $Q \ l \ r \ v$. What's the minimum index $i \in [l, r)$ such that $a[i] > v$, or -1 if no such index exists.
- **Input Format:** First line, n, q . Next q lines describe the operations.
- **Constraints:** $1 \leq n, q \leq 100,000$.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Sample Input:

4 7

U 0 2

U 1 3

Q 0 4 2

Q 0 4 3

Q 0 4 1

U 0 4

Q 0 4 2

Sample Output:

1

-1

0

0

Extension

- To guide our search we need to know whether a range contains a value that is at least v .
- For this, it suffices to store the max of each range.
- We know how to maintain this, it's just a point update, range query range tree.
- Now to find a value that is at least v we just need to search only nodes with $\max[i] > v$ and terminate our search once we have found a value.
- To find the first such i , just always search the left child's subtree first.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

Implementation Details:

- So far we've only recursed into nodes we need to by checking before recursing. For this it is a bit easier to always recurse and return immediately if we've recursed into a node whose range is disjoint from the query range.
- To find an index we have to recurse down to the leaves. So we no longer early exit when the query range is the same as the node's range.
- Instead we early terminate once we have found a leaf. To support this, our recursion will return a boolean indicating if we have found an index.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

```
using namespace std;

const int N = 100100;
int maxrt[1<<18];
int n;

// Standard max range tree.
void update(int p, int v, int i = 1, int cL = 0, int cR = n) {
    if (cR - cL == 1) {
        maxrt[i] = v;
        return;
    }
    int mid = (cL + cR) / 2;
    if (p < mid) update(p, v, i*2, cL, mid);
    else update(p, v, i*2+1, mid, cR);
    maxrt[i] = max(maxrt[i*2], maxrt[i*2+1]);
}
```

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

```
bool query_rec(int qL, int qR, int v, int &foundPlc, int i = 1, int cL = 0, int
               cR = n) {
    // Query range does not intersect the node's range.
    if (qL >= cR || qR <= cL) return false;
    // Nothing in i's range is big enough
    if (maxrt[i] <= v) return false;
    if (cR - cL == 1) {
        foundPlc = cL;
        return true;
    }
    int mid = (cL + cR) / 2;
    if (query_rec(qL, qR, v, foundPlc, i*2, cL, mid)) return true;
    if (query_rec(qL, qR, v, foundPlc, i*2+1, mid, cR)) return true;
    return false;
}

int query(int qL, int qR, int v) {
    int ans = -1;
    query_rec(qL, qR, v, ans);
    return ans;
}
```

Extension

- **Complexity?** Actually still $O(\log n)$ per operation.
- Recall our previous recursions stopped whenever we encountered a node whose range was entirely contained in $[qL, qR)$.
- In this recursion, whenever we encounter such a node, either its max value is too low and we stop anyways, or the node contains the index we are looking for.
- The latter case only occurs once and the search for the index is $O(\log n)$ since we only recurse from a node if we know for sure its range contains the desired index.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- This trick is useful for finding if an event has occurred in the array.

Extension

● Problem Statement:

It is 2155 and Earth has been renamed Water. LineLand with its constant showers has been particularly devastated. LineLand consists of n towns in a row, each with a height h_i . Initially all of these have water level 0.

The climatologists of LineLand forecast there will be m showers, the i -th raising the water levels of towns $[l_i, r_i)$ by w_i .

The mayor of LineLand wants to know how many towns are underwater (total water level is greater than the height of the town) after each shower.

● Input Format: First line, 2 integers, n, m .

$1 \leq n, m \leq 500,000$. Next line, n integers, the initial heights of the towns. Next m lines each describe a shower.

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Sample Input:

```
3 2
1 4 2
0 2 3
1 3 2
```

Sample Output:

```
1
2
```

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- **Observation 1:** Once a town is underwater, it is always underwater.
- So we just need to find out what towns change from above water to underwater after each operation.
- What is the criterion for a town to be underwater?
- That $\text{total_water}[i] > \text{height}[i]$.
- Alternatively that $0 > \text{height}[i] - \text{total_water}[i]$.

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- So to know if there is a new town underwater, we just need to know if $\min_i(\text{height}[i] - \text{total_water}[i]) < 0$.
- We can then delete this town so we do not count it more than once then repeat.
- For this problem, setting a town's height to infinity is as good as deleting the town.

Extension

```
#include <iostream>
using namespace std;

const int N = 500500;
const int INF = 1000*1000*1000+7; // large height to never go underwater
int minrt[1<<20]; // standard range update min range tree
int n, m;

void update(int uL, int uR, int v); // standard function for a[uL, uR) += v
int query(int qL, int qR, int v); // returns index of any value < 0, or -1 if
    none exist

int main(void) {
    cin >> n >> m;
    for (int i = 0; i < n; i++) {
        int cH;
        cin >> cH;
        update(i, i+1, cH);
    }

    int ans = 0;
    for (int i = 0; i < m; i++) {
        int a, b, w, cInd;
        cin >> a >> b >> w;
        update(a, b, -w);
        while ((cInd = query(0, n, 0)) != -1) {
            ans++;
            update(cInd, cInd+1, INF); // "delete" cInd
        }
        cout << ans << '\n';
    }
}
```

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

More on
Range Trees
RUPQ
RURQ
Range Trees
of DS
Searching a
Range Tree
DP
Optimizations
CHT
Construction
Application
Examples
D&C
Framework
Monotonicity
Modifications

- 1 More on Range Trees
 - Range Updates, Point Queries
 - Range Updates, Range Queries
 - Range Trees of Data Structures
 - Searching a Range Tree
- 2 DP Optimizations
 - Convex Hull Trick
 - Construction
 - Application
 - Examples
 - Divide and Conquer Optimization
 - Divide and Conquer Framework
 - Proving monotonicity of opt
 - Modifications

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- Sometimes when you're doing DP your solution will be too slow but your recurrence is very structured.
- You've seen this before with ranges and range trees.
- This lecture will show a few more cases of structured recurrences where there are well known techniques for speeding them up.
- Again, this does not help if your state space is too large. This should not really change your overall approach to DP but you should keep an eye out for recurrences that can be sped up.

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- One of the easiest to apply and most useful optimizations.
- Handy when your recurrence is formed by linear functions.
- Though once you get good at spotting these, more things look like a linear function than you might expect!

Extension

- Suppose you are doing a 1D DP, calculating $dp[1..M]$ in order of increasing i .
- Suppose you also have:
 - an array $m[1..M]$ in *decreasing order* - think of these as gradients, and
 - an array $p[1..M]$ - think of these as the positions of the points for which you are calculating $dp[\cdot]$.
- You have some base case value $dp[0]$ and the recurrence is

$$dp[i] = \min_{j < i} (dp[j] + m[j] \cdot p[i]) .$$

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- Take a look at the recurrence

$$dp[i] = \min_{j < i} (dp[j] + m[j] \cdot p[i]) .$$

- Hopefully you see an $O(N^2)$ solution.
- What do those terms in the min look like?
- Equations for lines with $dp[j]$ as the y-intercept and $m[j]$ as the slope.

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- Recurrence:

$$dp[i] = \min_{j < i} (dp[j] + m[j] \cdot p[i])$$

- Suppose so far we have

$$dp[0] = -2,$$

$$m[0] = 2$$

$$dp[1] = -1,$$

$$m[1] = 1$$

$$dp[2] = 5,$$

$$m[2] = -1$$

and we want to calculate $dp[3]$ (leave $p[3]$ unfixed for now).

Extension

- Recurrence:

$$dp[i] = \min_{j < i} (dp[j] + m[j] \cdot p[i])$$

- Then $dp[3]$ is the minimum of a set of lines at the x-coordinate $p[3]$, where our lines are:

$$y = m[0]x + dp[0] = 2x - 2$$

$$y = m[1]x + dp[1] = x - 1$$

$$y = m[2]x + dp[2] = -x + 5$$

- So far we have calculated each value $dp[i]$ in $O(N)$, by iterating through all of these lines. We will see how to speed this up.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DSSearching a
Range TreeDP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

- To speed up this recurrence, we will build a data structure that exactly supports the operations we need.
- It will export 2 methods:
 - `void add(int m, int b)`: Add a line $\ell = m \cdot x + b$.
Requirement: m is strictly less than the gradient of all lines in the data structure already.
 - `int query(q)`: Over all lines $\{\ell_i = m_i x + b_i\}$ that we have added so far, return

$$\min_i (m_i \cdot q + b_i).$$

- We will have add run in $O(1)$ *amortized* and query run in $O(\log n)$ where n is the number of lines added so far.
- Afterwards we will apply this data structure as a black box to some DP problems.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DSSearching a
Range TreeDP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

- To get the right complexity, we need some observations first regarding the set of lines we are querying.
- For concreteness, let us return to our earlier example. Suppose we have

$$\begin{array}{ll} b[0] = -2, & m[0] = 2 \\ b[1] = -1, & m[1] = 1 \\ b[2] = 5, & m[2] = -1 \end{array}$$

where $m[i]$ are the gradients and $b[i]$ are the y -intercepts.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

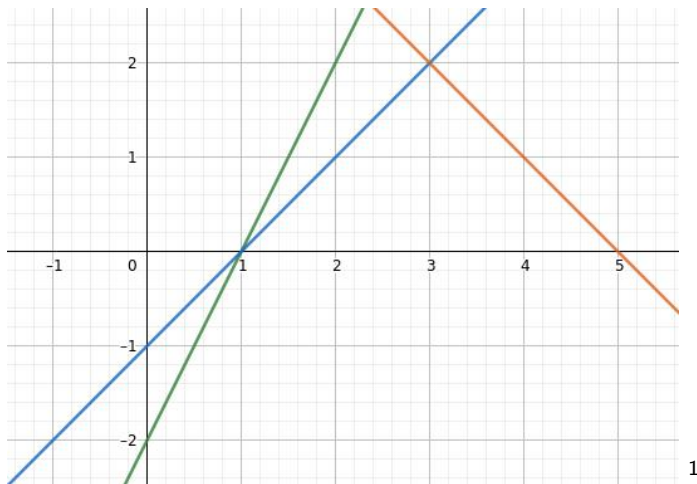
Examples

D&C

Framework

Monotonicity

Modifications



¹<https://www.geogebra.org/graphing>

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- Remember our aim is to query the minimum over these 3 lines at the x -coordinate q .
- What could this possibly be?
- Let's emphasize what the minimum value is at each x -coordinate.

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

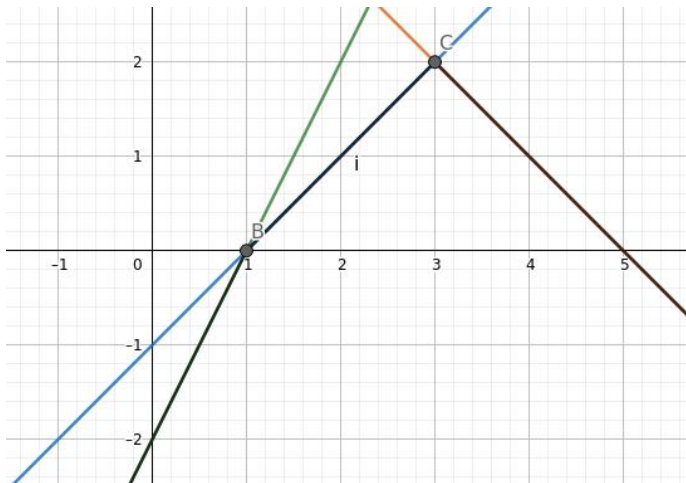
Examples

D&C

Framework

Monotonicity

Modifications



2

²<https://www.geogebra.org/graphing>

Extension

- **Key Observation:** This is the upper convex hull of the areas below the lines.
- This means each line is optimal for a contiguous range of x (possibly empty) and as we move left to right, the gradient of the optimal line never increases.
- Our goal is to maintain the convex hull by maintaining the set of line segments that the convex hull is made out of.
- We will say a line ℓ is dominant at x -coordinate x if it is the line that forms the convex hull at x , i.e. the line that gives the minimum in the equation

$$\text{query}(q) = \min_i (m_i \cdot q + b_i) .$$

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- For this, it suffices to store the lines that make up the convex hull in left to right order, i.e. in decreasing order of gradient.
- **Note:** Importantly, this does not contain all the lines. It omits any line that is never in the upper convex hull.
- Given this data, we can calculate the range of x at which each line ℓ is dominant.
- The segment is the range between the intersection of ℓ with the line before it and the line after it in the convex hull.
- We keep the lines in a vector.

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

```

struct line { long long m, b; };
double intersect(line a, line b) {
    return (double)(b.b - a.b) / (a.m - b.m);
}
// Invariant: cht[i].m is in decreasing order.
vector<line> cht;
/*
 * The intersection points are
 * intersect(cht[0], cht[1]), intersect(cht[1], cht[2]), ...
 * Line i is optimal in the range
 * [intersect(cht[i-1], cht[i]), intersect(cht[i], cht[i+1])]
 * where for i = 0, the first point is -infinity,
 * and for i = N-1, the last point is infinity.
 */
    
```

Extension

- Recall the 2 methods our data structure is meant to support are:

- What's the line on the upper convex hull at $x = q$?
Equivalently, what is

$$\min_i (m_i \cdot q + b_i)$$

over all the lines?

- Add the line

$$y = m[i] \cdot x + dp[i].$$

- We handle the former with binary search.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- Our aim is to find the line that is dominant at $x = q$.
- Suppose our lines are ordered in decreasing gradient order. Then recall the range at which line ℓ_i is dominant is

$$[\text{intersect}(\ell_{i-1}, \ell_i), \text{intersect}(\ell_i, \ell_{i+1})]$$

where for $i = 0$, the left term is $-\infty$ and for the last line, the right term is $+\infty$.

- So to find the i for which the segment contains q , it suffices to find the minimum i such that

$$\text{intersect}(\ell_i, \ell_{i+1}) \geq q.$$

Extension

```

struct line { long long m, b; };
double intersect(line a, line b) {
    return (double)(b.b - a.b) / (a.m - b.m);
}
// Invariant: cht[i].m is in decreasing order.
vector<line> cht;

/* Recall that the range the ith line is dominant in is:
 * [intersect(cht[i-1], cht[i]), intersect(cht[i], cht[i+1])]
 * We want to find the line that is dominant at x.
 * To do this, we note that the sequence (intersect(cht[i], cht[i+1]))
 * is monotonically increasing in i.
 * Hence we can binary search for the minimum i such that
 * intersect(cht[i], cht[i+1]) >= x
 */
long long query(long long x) {
    int lo = 0; int hi = cht.size()-2;
    // Find largest idx such that x <= intersect(cht[idx], cht[idx+1])
    // If this doesn't exist then idx should be cht.size()-1.
    int idx = cht.size()-1;
    while (lo <= hi) {
        int mid = (lo+hi)/2;
        if (intersect(cht[mid], cht[mid+1]) >= x) {
            idx = mid; hi = mid-1;
        } else { lo = mid+1; }
    }
    return cht[idx].m*x + cht[idx].b;
}
    
```

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

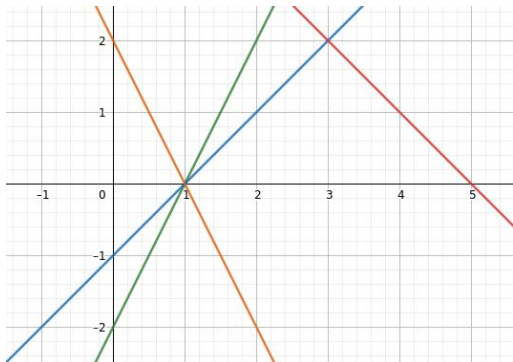
Monotonicity

Modifications

- To add a line we crucially use the fact that $m[1..M]$ is in decreasing order.
- Since the convex hull is sorted in non-increasing order of gradients, the new line has to go on the end of our convex hull.
- However, this may cause some lines to disappear from the convex hull.

Extension

- For example, consider adding the line $y = -2x + 2$ to the earlier example:



- What is the new convex hull?

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- So some of the lines may become useless and we need to remove them.
- When does a line ℓ become useless? When the line we just added covers the entire range ℓ is dominant in.
- **Observation:** The useless lines are always at the end of the convex hull.
- So we just need to keep popping the last line of the convex hull as long as it is useless.
- How do we check if the last line is useless?
- For this, it helps to draw pictures and move your new line ℓ around.

Extension

- You will hopefully observe some variant of the following:

The last line $cht[N-1]$ will be useless if

$$\text{intersect}(cht[N-1], \ell) \leq \text{intersect}(cht[N-2], cht[N-1]).$$

- Recall that $cht[N-1]$ is dominant in the range $(\text{intersect}(cht[N-2], cht[N-1]), \infty)$.
- The above essentially says ℓ is better than $cht[N-1]$ for this entire range.
- Another way of phrasing this is that the intersections $(\text{intersect}(cht[i], cht[i+1]))$ need to be kept in increasing order.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

```

struct line { long long m, b; };
double intersect(line a, line b) {
    return (double)(b.b - a.b) / (a.m - b.m);
}
// Invariant: cht[i].m is in decreasing order.
vector<line> cht;

void add(line l) {
    auto n = cht.size();
    while (n >= 2 &&
           intersect(cht[n-1], cht[n-2]) >= intersect(cht[n-1], l)) {
        cht.pop_back();
        n = cht.size();
    }
    cht.push_back(l);
}

long long query(long long x) {
    int lo = 0; int hi = cht.size()-2;
    // Find largest idx such that x <= intersect(cht[idx], cht[idx+1])
    int idx = cht.size()-1;
    while (lo <= hi) {
        int mid = (lo+hi)/2;
        if (intersect(cht[mid], cht[mid+1]) >= x) {
            idx = mid; hi = mid-1;
        } else { lo = mid+1; }
    }
    return cht[idx].m*x + cht[idx].b;
}
    
```

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- **Complexity?** $O(1)$ *amortized* per add. $O(\log n)$ per query.
- But do *remember*, we did assume the gradients are in decreasing order. It is non-trivial to remove this assumption.
- In certain special cases, we can actually get a better complexity!

Extension

- One common case is when we make queries in non-decreasing order.

- Formally, say our code calls query with

$$\text{query}(q_1), \text{query}(q_2), \dots, \text{query}(q_Q).$$

Then $q_1 \leq q_2 \leq \dots \leq q_Q$.

- In this case, the index of the dominant line for each q_i only increases.
- So we keep track of the index of the dominant line. Whenever we query, we check if the current dominant line is still the dominant line for the new query point.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

- If cp is the current dominant line, this amounts to checking if

$$p[i] \leq \text{intersect}(cht[cp], cht[cp + 1]).$$

- While this does not hold, increase cp .
- The above had an omission - there is a special case. Since the number of lines in the convex hull can decrease, cp may be out of bounds. To fix this, after each update we just need to update cp to point to the last line if it is out of bounds.
- If you want to rigorously check this, the invariant you are maintaining is

$$\text{intersect}(cht[cp - 1], cht[cp]) \leq p[i].$$

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

```
struct line { long long m, b; };
double intersect(line a, line b) {
    return (double)(b.b - a.b) / (a.m - b.m);
}
// Invariant: cht[i].m is in decreasing order.
vector<line> cht;
int cp;

void add(line l) {
    auto n = cht.size();
    while (n >= 2 &&
           intersect(cht[n-1], cht[n-2]) >= intersect(cht[n-1], l)) {
        cht.pop_back();
        n = cht.size();
    }
    cht.push_back(l);
    cp = min(cp, (int)cht.size()-1);
}

long long query(long long x) {
    while (cp+1 != cht.size() &&
           intersect(cht[cp], cht[cp+1]) < x) cp++;
    return cht[cp].m*x + cht[cp].b;
}
```

- **Complexity?** Now updates and queries are both $O(1)$ amortized.

Extension

- Currently query returns the minimum over all lines.
- We could also instead return the maximum.
- If so, our invariant is that our gradients should be increasing.
- We are now calculating the lower convex hull of the area above all lines.
- But we can actually use the exact same code! (literally no modifications needed)

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

- Not hard to adjust for gradients non-increasing (instead of decreasing) - just one extra special case.
- Can also have no conditions on the gradients. We still keep lines in sorted gradient order.
- However, we now need to keep lines in a set since insertions are arbitrary.
- Not that common mostly because it is tedious to get right.
- Alternatively we can also use a “Li Chao tree”.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

- Another useful modification is when everything (gradients and query points) is in integers.
- Currently we use doubles to compute the intersection points. This gives us the precise ranges at which we dominate (up to precision errors)
 $[\text{intersect}(\text{cht}[i-1], \text{cht}[i]), \text{intersect}(\text{cht}[i], \text{cht}[i+1])]$.
- Doubles are not precise though even for the range of a long long. Usually not problematic for competitions since numbers generally only go up to 10^9 but you have to keep it in mind if you need to query higher.
- If we only care about integer coordinates of x then we can round these down and work entirely in integers:

$$(\lfloor \text{intersect}(\text{cht}[i-1], \text{cht}[i]) \rfloor, \lfloor \text{intersect}(\text{cht}[i], \text{cht}[i+1]) \rfloor)$$

More on
Range Trees

RUPQ

RURQ

Range Trees
of DSSearching a
Range TreeDP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- Just **be careful**. If you want to do this, **make sure** it is clear to you why the above is exclusive-inclusive.
- In integers, whether your inequalities are strict or not actually matters. When you compare with an intersection point, mentally check if it agrees with the above ranges.
- If we only care about *positive* integers, we can omit the floor and just use regular integer division.
- But if you care about negatives, **note that** integer division isn't floor; it rounds towards 0.

Extension

```

struct line { long long m, b; };
long long floordiv(long long a, long long b) {
    return a / b - (a%b && ((a<0) ^ (b<0)));
}

long long intersect(line a, line b) {
    floordiv(b.b - a.b, a.m - b.m);
    // for POSITIVE ints: can do:
    // return (b.b - a.b) / (a.m - b.m);
}

vector<line> cht;
void add(line l) {
    auto n = cht.size();
    while (n >= 2 &&
           intersect(cht[n-1], cht[n-2]) >= intersect(cht[n-1], l)) {
        cht.pop_back();
        n = cht.size();
    }
    cht.push_back(l);
}

long long query(long long x) {
    int lo = 0; int hi = cht.size()-2;
    int idx = cht.size()-1;
    while (lo <= hi) {
        int mid = (lo+hi)/2;
        // NOTE: It is critical here that this is >= not >.
        if (intersect(cht[mid], cht[mid+1]) >= x) {
            idx = mid; hi = mid-1;
        } else { lo = mid+1; }
    }
    return cht[idx].m*x + cht[idx].b;
}
    
```

Extension

- The above construction gives us exactly the data structure we needed.
- As a reminder, it supports 2 methods:
 - `void add(int m, int b)`: Add a line $l = mx + b$.
Requirement: m is strictly less than the gradient of all lines in the data structure already.
Complexity: $O(1)$.
 - `int query(q)`: Over all lines $\{\ell_i = m_i \cdot x + b_i\}$ that we have added so far, return

$$\min_i (m_i \cdot q + b_i).$$

Complexity: $O(\log n)$ in general, we can make it $O(1)$ if queries are given in non-decreasing order.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

- There are 2 knobs we can tweak for our data structure.
 - If queries are in non-decreasing order, we can replace our query code with an $O(1)$ routine.
 - If everything is in integers, we can use integer division.
- However, aside from these, we can essentially use it as a black box!
- So the construction is good to know, but application-wise, you can just copy paste it in if need be.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DSSearching a
Range TreeDP
Optimizations

CHT

Construction
Application

Examples

D&C

Framework

Monotonicity

Modifications

- Let us return to our earlier DP problem. We had the recurrence

$$dp[i] = \min_{j < i} (dp[j] + m[j] \cdot p[i])$$

where we assumed the $m[j]$ are in decreasing order.

- Since the gradients are decreasing, we fulfil the one requirement of our CHT data structure.
- Hence we can directly apply convex hull trick as a black box.

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

```
int main() {  
    // Base case:  
    dp[0] = baseCaseCost;  
    line l;  
    l.m = m[0];  
    l.b = dp[0];  
    add(l);  
  
    for (int i = 1; i < N; i++) {  
        dp[i] = query(p[i]);  
        line l;  
        l.m = m[i];  
        l.b = dp[i];  
        add(l);  
    }  
    return 0;  
}
```

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- **Complexity?** $O(N \log N)$.
- This is a significant improvement from $O(N^2)$!
- Note, we could even get $O(N)$ if our $p[i]$ are non-decreasing.

Extension

- DPs with recurrences in the form

$$dp[i] = \min_{j < i} (dp[j] + m[j] \cdot p[i])$$

can be done in $O(N \log N)$.

- Use a data structure for CHT which supports:
 - `void add(int m, int b)`: Add a line $l = m \cdot x + b$.
Requirement: m is strictly less than the gradient of all lines in the data structure already.
 - `int query(q)`: Over all lines $\{l_i = m_i \cdot x + b_i\}$ that we have added so far, return

$$\min_i (m_i \cdot q + b_i).$$

- You can treat this data structure as a black box. However it is good to at least know how to tweak it using the 2 knobs mentioned above.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

In practice, the difficulty of CHT comes from:

- Recognizing that it is useful.
 - Often you just have to write out the recurrences.
 - Be suspect whenever the recurrence is given by a formula.
- Figuring out how to calculate the gradients and y -intercepts.
 - You will often have to juggle around terms in the recurrence to make it work.
 - This comes mostly with practice.
- Essentially, the recurrence

$$dp[i] = \min_{j < i} (dp[j] + m[j] \cdot p[i])$$

tells us that we need the gradient to be a function of only j and the query point to be a function of only i .

Extension

- **Problem Statement:**

You have to cover N points on a walkway.

To cover the walkway from point x to point y inclusive costs $C + (x - y)^2$ where C is a fixed, given constant.

Note that you can cover a single point with cost C .

What is the minimum cost needed to cover all the points?

- **Input Format:**

First line 2 integers, $N, C, 1 \leq N \leq 10^6, 1 \leq C \leq 10^9$.

The next N lines contain the points in increasing order.

All points are in the range $[1, 10^9]$.

- **Source:** 2012 University of Chicago Invitational Programming Contest.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

- Hope you can see a $O(N^2)$ DP.
- Calculate $dp[1..N]$, where $dp[i]$ is the min cost to cover exactly the points up to the i th.
- But the recurrence here is a very nice formula. Let us unpack it.

$$\begin{aligned} dp[j] &= \min_{i < j} (dp[i-1] + C + (x[j] - x[i])^2) \\ &= \min_{i < j} (dp[i-1] + C + x[i]^2 + x[j]^2 - 2 \cdot x[i] \cdot x[j]) \end{aligned}$$

- Focus on the last term. What does this look like?
- A linear function (if you squint hard enough)! So we should try to fit this into our CHT framework.

Extension

- Recurrence:

$$dp[j] = \min_{i < j} (dp[i - 1] + C + x[i]^2 + x[j]^2 - 2 \cdot x[i] \cdot x[j]) .$$

- If you recall, we were earlier looking at recurrences of the form

$$\min_{i < j} (dp[i] + m[i] \cdot p[j]) .$$

- What should $m[i]$ and $p[j]$ be?
- We should have $m[i] = -2x[i]$ and $p[j] = x[j]$, since we need the gradient to be determined by i and the query point by j .
- Note that our gradients are decreasing, which is required for CHT!

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

- Recurrence:

$$dp[j] = \min_{i < j} (dp[i - 1] + C + x[i]^2 + x[j]^2 - 2 \cdot x[i] \cdot x[j]) .$$

- We also need to modify the y-intercept of the line; it isn't just $dp[i]$ anymore. What should it be?
- **Key:** It needs to include all terms dependent on i . So it needs to include $dp[i - 1]$ and $x[i]^2$. **Why?**
- But it can't include $x[j]^2$ for obvious reasons. So we should add the $x[j]^2$ part when we calculate $dp[j]$.
- You can choose whether to add C when you calculate $dp[j]$ or instead add C to the y-intercept.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

- Recurrence:

$$dp[j] = \min_{i < j} (dp[i - 1] + C + x[i]^2 + x[j]^2 - 2 \cdot x[i] \cdot x[j]) .$$

- So for each i we will add a line with y -intercept

$$dp[i - 1] + x[i]^2$$

and gradient $-2x[i]$.

- To calculate $dp[j]$, we query our CHT with point $x[j]$. Define $r := \text{query}(x[j])$. Then

$$dp[j] = r + C + x[j]^2 .$$

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

```

/* Insert CHT code here: You can use the version with only
 * positive integer queries and non-decreasing queries */
void add(line l);
long long query(long long x);

const int MAXN = 1000000;
int N;
long long x[MAXN+1];
long long C;

long long dp[MAXN+1];

int main() {
    scanf("%d %lld", &N, &C);
    for (int i = 0; i < N; i++) {
        scanf("%lld", &x[i]);
    }
    for (int i = 0; i < N; i++) {
        // Compare to formulas written in slides
        dp[i] = query(x[i]) + C + x[i]*x[i];
        line l;
        l.m = -2*x[i];
        // Base case is i == 0, dp[i-1] = 0.
        l.b = (i == 0 ? 0 : dp[i-1]) + x[i]*x[i];
        add(l);
    }
    // Again, calculating dp[N-1] using the same formula as above.
    printf("%lld\n", dp[N-1]);
}

```

Extension

More on
Range Trees
RUPQ
RURQ
Range Trees
of DS
Searching a
Range Tree
DP
Optimizations
CHT
Construction
Application
Examples
D&C
Framework
Monotonicity
Modifications

- **Complexity?** We note our query points $p[j] = x[j]$ are increasing in j , so we can use the $O(1)$ amortized CHT. Then the complexity is just $O(N)$.
- Since our query points are integers, we can also use the version of CHT with no doubles.
- Using just integer division doesn't change the complexity but in practice is a large speed up (I get a 3 times speedup locally).

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

This is how many CHT problems go.

- First you come up with a normal DP that is too slow.
This step is the same as in the previous DP lecture.
- Then you note the recurrence is a nice formula.
Write it out.
- Now, split up the recurrence as in this example.
Figure out which part corresponds to the slope and query point and how to split up the constant into the y -intercept and the part added when you calculate $dp[j]$.
- Usually this is just breaking up the terms based on if they depend on j or on i .

Extension

● **Problem Statement:**

You have N soldiers in a line, the i th of which has effectiveness x_i .

You want to partition the soldiers into squadrons. Each squadron consists of a subsegment of *contiguous* soldiers. Say these soldiers have total effectiveness S . Then the effectiveness of the squadron is $AS^2 + C$ where $A < 0$ and C are given constants.

Maximize the sum of the effectiveness of the squadrons.

● **Input Format:**

First line contains three integers, N, A, C .

$1 \leq N \leq 10^6, -5 \leq A \leq -1, |C| \leq 10^4$.

Next line contains N integers, the effectiveness of the soldiers in order.

Each effectiveness satisfies $1 \leq x_i \leq 100$.

● **Source:** APIO 2010.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction
Application

Examples

D&C

Framework

Monotonicity

Modifications

- **Sample Input:**

4 -1 -20

2 2 3 4

- **Sample Output:** -101

- **Explanation:** Split the soldiers into groups of $\{1, 2\}$, $\{3\}$, $\{4\}$, with effectiveness $\{-36, -29, -36\}$.

Extension

- Hope you can see a $O(N^2)$ DP.
- Calculate $dp[1..N]$, where $dp[i]$ is the min cost to partition exactly the first i soldiers.

- Where to go next?
- Look at the recurrence!

$$dp[j] = \max_{i < j} (dp[i - 1] + C + A(x_i + x_{i+1} + \dots + x_j)^2)$$

- This is kind of messy but should remind you a bit of the last example.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

- **Useful trick:** We want to clean up the sum in the square. It is often easier to work with cumulative arrays than sums.
- Define $S[M]$ to be the cumulative sum, $S[i] := (x_1 + \dots + x_i)$.
- Then let's rewrite the recurrence.

$$\begin{aligned} dp[j] &= \max_{i < j} (dp[i-1] + C + A(S[j] - S[i-1])^2) \\ &= \max_{i < j} (dp[i-1] + C + A \cdot S[i-1]^2 + A \cdot S[j]^2 \\ &\quad - 2A \cdot S[i-1] \cdot S[j]) \end{aligned}$$

- This gives a much cleaner separation between terms in i and terms in j .

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Recurrence:

$$dp[j] = \max_{i < j} (dp[i-1] + C + A \cdot S[i-1]^2 + A \cdot S[j]^2 - 2A \cdot S[i-1] \cdot S[j])$$

- What does this look like?
- The key is the last term. If you unpack the important parts, a line!

Extension

- Again, main question is to convert this into $b[i] + m[i] \times p[j]$ form. Let us do the latter part first.
- What is $m[i]$? $-2A \cdot S[i - 1]$. So $p[j] = S[j]$. Gradient is increasing (since A is negative), which is correct for maximization problems.
- How about the constant? Which parts form $b[i]$, and which parts should we add directly to $dp[j]$?
- Again, we need $b[i]$ to include $dp[i - 1] + A \cdot S[i - 1]^2$. This time around we will also include C in $b[i]$.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction
Application

Examples

D&C

Framework
Monotonicity
Modifications

Extension

Recurrence:

$$dp[j] = \max_{i < j} (dp[i-1] + C + A \cdot S[i-1]^2 + A \cdot S[j]^2 \\ (-2A \cdot S[i-1] \cdot S[j]))$$

- So for i we will add a line with y -intercept

$$dp[i-1] + A \cdot S[i-1]^2 + C$$

and gradient $-2A \cdot S[i-1]$.

- To calculate $dp[j]$, we query our CHT with point $S[j]$. Define $r := query(S[j])$. Then

$$dp[j] = r + A \cdot S[j]^2.$$

More on
Range Trees

RUPQ

RURQ

Range Trees
of DSSearching a
Range TreeDP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

```

/* Insert CHT code here: You can use the version with only
 * positive integer queries and non-decreasing queries */
void add(line l);
long long query(long long x);

const int MAXN = 1000000;
int N;
long long a, c;
long long S[MAXN+1];
long long dp[MAXN+1];

int main() {
    scanf("%d", &N);
    scanf("%lld %lld %lld", &a, &c);
    for (int i = 1; i <= N; i++) {
        long long x; scanf("%lld", &x);
        S[i] = S[i-1] + x;
    }
    for (int i = 1; i <= N; i++) {
        line l;
        l.m = -2*a*S[i-1];
        l.b = dp[i-1] + a*S[i-1]*S[i-1] + c;
        add(l);
        dp[i] = query(S[i]) + a*S[i]*S[i];
    }
    printf("%lld\n", dp[N]);
    return 0;
}

```

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- This time around we want to maximize the value. However, our code for CHT works for both min (with decreasing gradients) and max (with increasing gradients) without modification.
- **Complexity?** We note our query points $p[j] = S[j]$ are increasing in j , so we can use the $O(1)$ amortized CHT. Then the complexity is just $O(N)$.
- One should be a bit cautious here of overflows. Here $S[i]$ can go up to 10^8 , but we take $S[i]^2$, and $dp[i]$ can go up to around 10^{13} . This is okay with long longs.
- Our y -intercepts go up to around 10^{16} , so we're going to lose precision if we use double for our intercepts. This seems to be OK since our query points only go up to 10^8 (it gets AC) but it's at least a concern.

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- Here the structure comes in the choices we make during the DP.
- The most common setting is a 2D DP.

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DSSearching a
Range TreeDP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- **Problem Statement:**

You have N kids in a row. Each has a noisiness $s_i > 0$.

You want to partition the kids into K rooms.

Each room must contain an *contiguous* segment of the kids.

The noisiness of a room containing kids $[i, j]$ is

$$(s_i + s_{i+1} + \dots s_j)^2.$$

What is the minimum total noisiness that is attainable?

- **Input Format:**

First line 2 integers, N, K .

$1 \leq K \leq \min(100, N), 1 \leq N \leq 10^4$.

Next line contains N integers, the noisiness of the kids in order.

Each noisiness is in the range $[1, 1000]$.

Extension

- Hope you can see an $O(N^2K)$ DP.
- Our state will be $dp[k][i]$, the minimum noisiness to partition the first i kids into k groups.
- We will proceed in increasing k and increasing i order.
- Recurrence is

$$dp[k][i] = \min_{j < i} (dp[k-1][j] + (s_{j+1} + \dots + s_i)^2).$$

- Naively this recurrence is $O(N^2)$, there are $O(N)$ values of j and the sum takes $O(N)$ to calculate. How do we speed up this recurrence to $O(N)$?
- Cumulative sum!

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

```
#include <bits/stdc++.h>
using namespace std;

const long long INF = 1e17;
const int MAXN = 10000;
const int MAXK = 100;
int N, K;
// 1-indexed. S[i] = sum(s_1 + ... + s_i)
long long S[MAXN+1];
long long dp[MAXK+1][MAXN+1];

int main() {
    scanf("%d %d", &N, &K);
    for (int i = 1; i <= N; i++) {
        long long s; scanf("%lld", &s);
        S[i] = S[i-1] + s;
    }
    for (int i = 1; i <= N; i++) dp[0][i] = INF;
    for (int k = 1; k <= K; k++) {
        for (int j = 1; j <= N; j++) {
            dp[k][j] = INF;
            for (int t = 0; t < j; t++) {
                dp[k][j] = min(dp[k][j],
                               dp[k-1][t] + (S[j]-S[t])*(S[j]-S[t]));
            }
        }
    }
    printf("%lld\n", dp[K][N]);
    return 0;
}
```

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

- Let us take a look at the recurrence again:

$$dp[k][i] = \min_{j < i} (dp[k-1][j] + (s_{j+1} + \dots + s_i)^2).$$

- Let $opt[k][i]$ be the argmin, i.e. the value of j at which the minimum is attained in the above recurrence. If there's several, pick any.

- Key Claim:**

$$opt[k][i] \leq opt[k][i+1]$$

- Proof later. Intuitively, one expects increasing i to move our groups to the right. If $opt[k][i+1] < opt[k][i]$ then the k th group for $i+1$ encompasses the k th group for i . This seems wrong because the cost of squaring increases quicker the larger the group gets.
- More formally, the key is x^2 is *convex*.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

- This is the setting we work in. We have a 2D dp with recurrence

$$dp[k][i] = \min_{j < i} (dp[k-1][j] + \text{Cost}(j+1, i)).$$

- We define the array $opt[k][i]$ to be any value of j at which the above attains a minimum.
- And we know (through some voodoo or maths) that

$$opt[k][i] \leq opt[k][i+1].$$

- We now see how to speed this up.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

The structure of the following slides:

- In the next few slides, I assume the setting just mentioned. In particular, I assume that $opt[k][i] \leq opt[k][i+1]$ for all k, i .
- Under this setting, I will explain the general procedure for optimising the DP from $O(N^2K)$ to $O(NK \log N)$.
- I will demonstrate the procedure on our example problem.
- I will then talk about when $opt[k][i] \leq opt[k][i+1]$ holds. As a special case, we will show it holds for our example problem.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

- Suppose $opt[k][i] \leq opt[k][i + 1]$.
- The earlier recurrence for $dp[k][i]$ was $O(N)$ because we had to try every $j < i$.
- But if we know $prev := opt[k][i - 1]$ then we only need to try every $j \in [prev, i)$.
- This doesn't lead to a speed up just yet. But for example, if we also know the value $nxt := opt[k][i + 1]$, then we only have to try every $j \in [prev, nxt]$.
- This is how we will proceed: by constraining the range we have to search on both sides.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

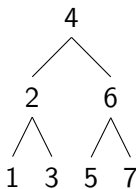
- First we calculate $dp[k][N/2]$ and $opt[k][N/2]$.
For convenience, define

$$o_{k,N/2} := opt[k][N/2].$$

- Now, for all $i < N/2$ we know $o_{k,i} < o_{k,N/2}$,
and for all $i > N/2$ we know $o_{k,i} > o_{k,N/2}$.
So on the left we have to search the range $[1, o_{k,N/2}]$ and
on the right $[o_{k,N/2}, M]$.
- Note: this partitions the range between the 2 halves.
- We now divide and conquer by repeating this procedure in
both halves, except only searching the above range in each
of the halves.

Extension

- Suppose we have k fixed and $N = 7$. Then we calculate $dp[k][n]$ in top to bottom order in the following tree.
- Each node will store its index i and $o_{k,i}$ in the form $\mathbf{o}:o_{k,i}$.
- Each node will also the range in which it searched for its recurrence like $\mathbf{[x, y]}$.



More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DSSearching a
Range TreeDP
Optimizations

CHT

Construction

Application

Examples

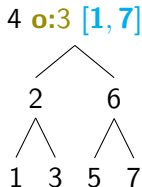
D&C

Framework

Monotonicity

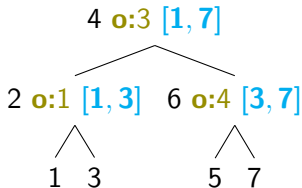
Modifications

- First, we have to search the range $[1, 7]$ to find $dp[k][4]$ and $o_{k,4}$. Suppose $o_{k,4} = 3$:



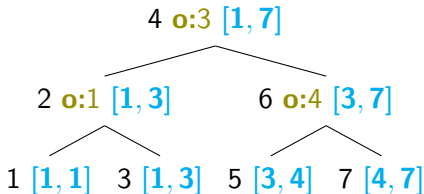
Extension

- Next we calculate $dp[k][i]$ for the 2 children. We should only search for j in the ranges that make sense. Suppose $o_{k,2} = 1$ and $o_{k,6} = 4$.



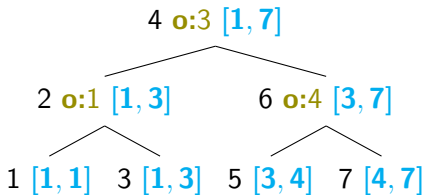
Extension

- Finally we calculate $dp[k][i]$ for the last layer, again only searching in the ranges that make sense. I am going to omit $\mathbf{o}:x$ for this layer because it does not matter.



Extension

- What can you say about the search ranges of each layer of the tree?



- Besides overlaps at endpoints, they form a partition of $[1, 7]$.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

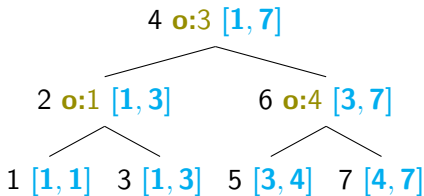
Framework

Monotonicity

Modifications

Extension

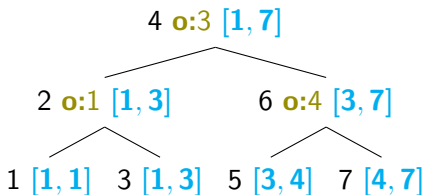
- Under this procedure, what is the cost to calculate $dp[k][2]$?



- The size of the range we have to search in for index 2. In this case it is $|[1, 3]| = 3$.

Extension

- What is the total cost to calculate all the $dp[k][i]$?



- Sum of the sizes of the ranges. The sum of the sizes for each layer is $O(N)$, there are $O(\log N)$ layers. Therefore $O(N \log N)$ overall.

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- Hence by calculating our DP in this divide and conquer order, we get a speed up of $O(N^2K)$ to $O(NK \log N)$.
- In practice, we DFS instead of doing it layer by layer.
- Our code is similar to range trees.
During our DFS, we keep track of a range $[cL, cR)$ and our DFS will calculate $dp[k][i]$ for all $i \in [cL, cR)$.
We also keep the search range (the range in the tree diagrams).

Extension

```

    Insert your cost function here:
    long long Cost(int i, int j);

    // Search range for our DP is in the range [qL, qR]
    // In this branch, calculating dp[k][cL,..cR)
    void dnc(int qL, int qR, int cL, int cR, int k) {
        if (cR <= cL) return;
        int bestpos(-1);
        int mid = (cL + cR) / 2;
        dp[k][mid] = INF; // assume this is a minimisation problem
        for (int i = qL; i <= min(qR, mid-1); i++) {
            // Depending on definition of Cost,
            // add Cost(i+1, mid) or Cost(i, mid)
            long long newcost = dp[k-1][i] + Cost(i+1, mid);
            if (newcost < dp[k][mid]) {
                dp[k][mid] = newcost;
                bestpos = i;
            }
        }
        // Split our range in 2.
        // In the left, the search range is [qL, bestpos]
        // and we calculate dp[k][cL,mid)
        // In the right, the search range is [bestpos, qR]
        // and we calculate dp[k][mid+1,cR)
        dnc(qL, bestpos, cL, mid, k);
        dnc(bestpos, qR, mid+1, cR, k);
    }

```

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- Let us return to our original example problem.
- What is the cost function here? Letting $S[M]$ be the cumulative array, we have

$$\text{Cost}(j+1, i) = (S[i] - S[j])^2.$$

- We can substitute this directly into our divide and conquer template.

Extension

```
#include <bits/stdc++.h>
using namespace std;

const long long INF = 1e17;
const int MAXN = 10000;
const int MAXK = 100;
int N, K;
long long S[MAXN+1]; // cumulative array
long long dp[MAXK+1][MAXN+1];

// Cost of segment (i, j]
long long Cost(int i, int j) {
    return (S[j]-S[i])*(S[j]-S[i]);
}

// Search range: [qL, qR], calculating dp[k][cL..cR]
void dnc(int qL, int qR, int cL, int cR, int k) {
    if (cR <= cL) return;
    int bestpos;
    int mid = (cL + cR) / 2;
    dp[k][mid] = INF;
    for (int i = qL; i <= min(qR, mid-1); i++) {
        // Cost expects [], so we use Cost(i, mid).
        long long newcost = dp[k-1][i] + Cost(i, mid);
        if (newcost < dp[k][mid]) {
            dp[k][mid] = newcost;
            bestpos = i;
        }
    }
    dnc(qL, bestpos, cL, mid, k);
    dnc(bestpos, qR, mid+1, cR, k);
}
```

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

```
int main() {
    scanf("%d %d", &N, &K);
    for (int i = 1; i <= N; i++) {
        long long c; scanf("%lld", &c);
        S[i] = S[i-1] + c;
    }
    // For K = 0, dp[0][0] = 0 is the base case.
    for (int i = 1; i <= N; i++) dp[0][i] = INF;
    // Just call dnc for k from 1 to N
    // Make sure you get the initial [qL, qR] and [cL, cR] correct.
    for (int k = 1; k <= K; k++) dnc(0, N, 1, N+1, k);
    printf("%lld\n", dp[K][N]);
    return 0;
}
```

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- **Complexity?** $O(NK \log N)$. Here $N = 10^4$, $K = 100$ we get $\approx 14 \cdot 10^6$ which is fine for a second.
- But we left out a detail: how do we know that $opt[k][i]$ is non-decreasing for a fixed k ?
- This is a special case of a more general phenomenon.

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- Warning: this part is quite technical.
I've included a key takeaways slide at the end.
- The more important part to get is the intuition.
- Feel free to skip verifying any of the maths equations.

Extension

- For concreteness, let $\text{Cost}(i, j) := (S[j] - S[i])^2$.

- I claim $\text{Cost}(i, j)$ satisfies the following key feature:
if $a < b < c < d$, then

$$\text{Cost}(a, d) - \text{Cost}(b, d) \geq \text{Cost}(a, c) - \text{Cost}(b, c).$$

- What does this say? Suppose we started by looking at the ranges $[b, c]$ and $[b, d]$ and we define $A := \text{Cost}(b, c)$ and $B := \text{Cost}(b, d)$.
- Then it says that adding an extra item to the interval $[b, d]$ increases the cost more than adding the same item to the interval $[b, c]$.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- But when our function is squaring, this key feature is immediately true. It just says that

$$(B + c)^2 - B^2 \geq (A + c)^2 - A^2$$

if $B > A \geq 0$.

- This follows from x^2 being convex (its derivative is an increasing function).

Extension

- Now, let Cost be any function that satisfies

$$\text{Cost}(a, d) - \text{Cost}(b, d) \geq \text{Cost}(a, c) - \text{Cost}(b, c)$$

for all $a < b < c < d$.

- To prove $\text{opt}[k][i]$ is increasing in i , we proceed with the standard swapping argument.
- Suppose for a contradiction that $\text{opt}[k][i+1] < \text{opt}[k][i]$.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

- Then on the one hand by definition of $o_{k,i+1}$:

$$\begin{aligned} dp[k-1][o_{k,i+1}] + \text{Cost}(o_{k,i+1}, i+1) \\ < dp[k-1][o_{k,i}] + \text{Cost}(o_{k,i}, i+1), \end{aligned}$$

or equivalently:

$$\begin{aligned} \text{Cost}(o_{k,i+1}, i+1) - \text{Cost}(o_{k,i}, i+1) \\ < dp[k-1][o_{k,i}] - dp[k-1][o_{k,i+1}]. \end{aligned}$$

- But the opposite situation holds for $o_{k,i}$:

$$\begin{aligned} dp[k-1][o_{k,i}] + \text{Cost}(o_{k,i}, i) \\ < dp[k-1][o_{k,i+1}] + \text{Cost}(o_{k,i+1}, i), \end{aligned}$$

or equivalently:

$$\begin{aligned} dp[k-1][o_{k,i}] - dp[k-1][o_{k,i+1}] \\ < \text{Cost}(o_{k,i+1}, i) - \text{Cost}(o_{k,i}, i). \end{aligned}$$

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- Composing these two inequalities, we get

$$\begin{aligned} & \text{Cost}(o_{k,i+1}, i+1) - \text{Cost}(o_{k,i}, i+1) \\ & < \text{Cost}(o_{k,i+1}, i) - \text{Cost}(o_{k,i}, i) \end{aligned}$$

exactly contradicting the desired inequality on Cost, since

$$o_{k,i+1} < o_{k,i} < i < i+1.$$



Extension

- This inequality

$$\text{Cost}(a, d) - \text{Cost}(b, d) \geq \text{Cost}(a, c) - \text{Cost}(b, c)$$

for all $a < b < c < d$ is called the quadrangle inequality.

- You should think of it as saying the larger your segment already is, the more costly it is to put items into it.
- So intuitively, one might expect this to imply monotonicity of opt , since moving the right endpoint from i to $i + 1$ should disincentivise the last segment growing any larger to the left.
- This is what the above proof showed. Hence, satisfying the quadrangle inequality is sufficient for opt to be monotonic, allowing us to apply D&C Optimisation.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

Some examples:

- Our example involved

$$\text{Cost}(i, j) = F(s_{i+1} + \dots + s_j)$$

where $F(x) = x^2$. If all our $\{s_i\}$ are positive then Cost satisfies the quadrangle inequality whenever F is convex. So $F(x) = x^3, x\sqrt{x}, x\log x$ all work.

- Also a lot of CHT, i.e.

$$\text{Cost}(i, j) = b_i + m_i \cdot p_j$$

where the m_i are non-increasing and the p_j are non-decreasing. Conversely, a decent number of D&C problems can be solved with CHT after clever rewriting.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- By induction, it suffices to prove it for one step:

$$\text{Cost}(b-1, c+1) - \text{Cost}(b, c+1) \geq \text{Cost}(b-1, c) - \text{Cost}(b, c).$$

- Often $\text{Cost}(i, j)$ is defined as some function operating on the multiset of $\{s_i, s_{i+1}, \dots, s_j\}$. For these, proving a single step is often easy.

- For example,

$$\text{Cost}(i, j) = |\{(a, b) \mid s_a + s_b = 0 \wedge a, b \in [i, j]\}|.$$

Extension

Key Takeaways:

- To show opt is monotone, we almost always instead show that Cost satisfies the quadrangle inequality.
- You don't need to know why Cost satisfying the quadrangle inequality implies monotonicity of opt . But it is worth knowing roughly how to check the inequality holds.
- Intuitively, it just says the larger a range already is, the more Cost increases more upon adding another item.
- Many Cost functions satisfy the quadrangle inequality, in particular convex functions applied to the sum of the range.
- If Cost satisfies the quadrangle inequality or we suspect it does, then we can apply the earlier Divide and Conquer framework directly just by modifying the Cost function.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

- For maximisation problems, we want the opposite quadrangle inequality

$$\text{Cost}(a, d) - \text{Cost}(b, d) \leq \text{Cost}(a, c) - \text{Cost}(b, c),$$

that is, it is harder to increase the cost of larger segments.

- Examples include concave functions such as $F(x) := \sqrt{x}$ and $F(x) = \log x$.
- Again, it suffices to prove it for a single step.
- Often the annoying part is calculating $\text{Cost}(i, j)$ quickly.
- For example, consider

$$\text{Cost}(i, j) = |\{(i, j) \mid s_i + s_j = 0\}|.$$

For D&C it is too slow to calculate this in $O(j - i)$.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- The quadrangle inequality further gives that

$$\text{opt}[k-1][i] \leq \text{opt}[k][i] \leq \text{opt}[k][i+1]$$

Using this and doing the DP in increasing k order, then decreasing i order, gives a running time of $O((N+K)N)$. This is sometimes called "Knuth's Optimisation". (though I feel this term is overloaded)

Extension

More on
Range Trees
RUPQ
RURQ
Range Trees
of DS
Searching a
Range Tree
DP
Optimizations
CHT
Construction
Application
Examples
D&C
Framework
Monotonicity
Modifications

- **Problem Statement:**

There are N mines on a river, of which the i th is at position X_i and produces W_i units of coal.

To collect coal, K facilities will be built on the river. The river only flows downstream so coal from position i can be delivered to position j iff $j > i$. To deliver w units of coal from position X to Y costs $w \cdot (Y - X)$ units of fuel.

If we optimally position the facilities, what is the minimum amount of fuel necessary to collect all the coal?

- **Source:** 2012 Latin America Regionals.

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- **Input Format:**

First line, 2 integers N, K . $1 \leq K < N \leq 2000$.

Next N lines describe the mines. Each line contains 2 integers X_i, W_i with $1 \leq X_i, W_i \leq 10^6$.

The mines are in strictly increasing order of X_i .

- **Sample Input:**

3 2

11 3

12 2

14 1

- **Sample Output:** 3

Extension

- As always, start simple. What order to process in and what state?
- The standard ones for 2D DP. State is $dp[k][i]$, the minimum cost to collect coal exactly from the first i mines using k facilities.
- Order is increasing k then increasing i . Increasing i then k works too.
- Recurrence? Has to be something about building facilities.
- **Easy Observation:** We should only build facilities at mines.
- By definition, in the state $dp[k][i]$, we must build a facility at mine i .
- So what is left to choose? Where the previous facility was built.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

- Recurrence:

$$dp[k][i] = \min_{j < i} (dp[k-1][j] + \text{Cost}(j, i))$$

What is $\text{Cost}(j, i)$ here? (in words)

- It is the total cost to deliver coal from mines $j+1, \dots, i$ to a facility at mine i .
- Formula is

$$\sum_{t=j+1}^i W_t \cdot (X_i - X_t).$$

- Exercise: Make this run in $O(N^2K)$ (Hint: Either rewrite the equation or do a backwards sweep to calculate the above min).
- $O(N^2K) \approx 8 \times 10^9$. Too slow (at least without extreme micro optimization).

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

- How to speed this up? Note this is a standard 2D DP where we are making a choice per state, but making this choice is introducing an $O(N)$ cost. And we are blowing out due to this $O(N)$ cost, not the state space.
- So we should look at whether this choice has a nice monotone structure, i.e. is $opt[k][i] \leq opt[k][i+1]$?
- To check this, we should check the quadrangle inequality, or the one step version

$$Cost(b-1, c+1) - Cost(b, c+1) \geq Cost(b-1, c) - Cost(b, c).$$

- **Important:** What does your intuition tell you?
- My intuition tells me this seems likely.
Adding another mine to a larger segment should incur more cost, compared to adding it to a smaller segment. In the former, the coal will have to be shipped further.

Extension

- We can also give a quick proof that the inequality

$$\text{Cost}(b-1, c+1) - \text{Cost}(b, c+1) \geq \text{Cost}(b-1, c) - \text{Cost}(b, c)$$

does indeed hold true.

- Recall that

$$\text{Cost}(j, i) := \sum_{t=j+1}^i W_t \cdot (X_i - X_t).$$

- Most terms of the inequality cancel. What remains on the LHS is just $W_b \cdot (X[c+1] - X[b])$, while RHS is just $W_b \cdot (X[c] - X[b])$.
- Therefore the quadrangle inequality is satisfied and we can apply D&C optimization.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

- The final ingredient is to calculate

$$\text{Cost}(j, i) := \sum_{t=j+1}^i W_t \cdot (X_i - X_t)$$

in constant time.

- The difficulty is that X_i varies, so we can't precompute this.
- Standard trick: rewrite this to isolate out the X_i part:

$$\text{Cost}(j, i) := \left(\sum_{t=j+1}^i W_t \right) X_i - \left(\sum_{t=j+1}^i W_t X_t \right).$$

- How do we calculate this in $O(1)$ now?
- Cumulative sums over the array $W[M]$ and the array $\{W_t X_t\}_{t=1}^N$.

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

```
#include <bits/stdc++.h>
using namespace std;
const long long INF = 1e18;
const int MAXN = 1000, MAXK = 1000;
int N, K;
long long X[MAXN+1], W[MAXN+1], sXW[MAXN+1], sW[MAXN+1];
long long dp[MAXK+1][MAXN+1];

// Cost of moving mines (i,j] to mine j.
long long Cost(int i, int j) {
    return (sW[j]-sW[i])*X[j] - (sXW[j]-sXW[i]);
}

// Best choice for our DP is in the range [qL, qR]
// Calculating dp[k][cL,..cR]
void dnc(int qL, int qR, int cL, int cR, int k) {
    if (cR <= cL) return;
    int bestpos(-1);
    int mid = (cL + cR) / 2;
    dp[k][mid] = INF;
    for (int i = qL; i <= min(qR, mid-1); i++) {
        // We use Cost(i, mid) since Cost(i,j) is the cost of (i,j]
        long long newcost = dp[k-1][i] + Cost(i, mid);
        if (newcost < dp[k][mid]) {
            dp[k][mid] = newcost;
            bestpos = i;
        }
    }
    dnc(qL, bestpos, cL, mid, k);
    dnc(bestpos, qR, mid+1, cR, k);
}
```

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

Extension

More on
Range Trees

RUPQ

RURQ

Range Trees
of DS

Searching a
Range Tree

DP
Optimizations

CHT

Construction

Application

Examples

D&C

Framework

Monotonicity

Modifications

```
int main() {
    scanf("%d %d", &N, &K);
    for (int i = 1; i <= N; i++) {
        scanf("%lld %lld", &X[i], &W[i]);
        sXW[i] = sXW[i-1] + X[i] * W[i];
        sW[i] = sW[i-1] + W[i];
    }
    for (int i = 1; i <= N; i++) dp[0][i] = INF;
    for (int k = 1; k <= K; k++) dnc(0, N, 1, N+1, k);
    printf("%lld\n", dp[K][N]);
    return 0;
}
```

Extension

More on
Range Trees
RUPQ
RURQ
Range Trees
of DS
Searching a
Range Tree
DP
Optimizations
CHT
Construction
Application
Examples
D&C
Framework
Monotonicity
Modifications

- **Complexity?** $O(NK \log N) \approx 4.4 \times 10^7$. Likely fine for a second on a relatively modern machine.
- Common in many problems of this kind.
 - Start with the obvious DP but the recurrence is too slow.
 - Look at the structure there. Since you are making choices, check if these choices might be monotone.
You want your intuition to play a strong role here.
 - To apply D&C, want Cost to be around $O(1)$ amortized.
For simple examples, this usually just involves rewriting the equation until you can apply basic data structures.
For more complicated examples, you may need to be more careful with how you persist state in your D&C.
- Aside: with some rewriting, not hard to do this with CHT either.